

Lineamiento de Nombramiento de Objetos de Base de Datos

Realizado por	Fecha	Firma
Anibal Hugo Cipriano Carrascal	07/01/2025	
Revisado por	Fecha	Firma
Cafu Junior Aguilar Baldeon		
Luis Miguel Salguero Sánchez		
Jonathan Franchesco Torres Baca		
Aprobado por	Fecha	Firma
José Jesús Salazar Silva		

1. INTRODUCCIÓN

2. ESTÁNDARES PARA OBJETOS DE BASE DE DATOS

3. DOCUMENTACIÓN

1. INTRODUCCIÓN

1.1 Objetivos

Definir los lineamientos generales para la creación de objetos de base de datos ligados a la construcción de programas en todos los Sistemas y/o Aplicativos de la empresa Atlantic CITY. Estos estándares contribuyen a la calidad, facilidad de entendimiento y mantenimiento de los Sistemas y/o Aplicativos.

1.2 Alcance

Estos lineamientos se aplican a todos los Sistemas y/o Aplicativos de la empresa Atlantic CITY. En este documento se definen los estándares para la creación de base de datos, esquemas, tablas, secuencias, procedimientos, tipos de datos, vistas, funciones y estructuras a utilizarse en el desarrollo de cualquier software desarrollado por la Institución.

2. ESTÁNDARES PARA OBJETOS DE BASE DE DATOS

2.1 Guías genéricas y Buenas prácticas

- No utilizar NCHAR, NVARCHAR, NTEXT. Solo aplicarlo para casos específicos, como por ejemplo: Cuando se desea almacenar información como el idioma chino y japonés.
- No dejar líneas obsoletas (líneas comentadas) dentro del cuerpo de un procedimiento almacenado, funciones, vistas u otros objetos de programación de base de datos.
- Utilizar en minúsculas los nombres de cualquier objeto de base de datos.
- Los nombres en general no deben de contener espacios en blanco. Si se necesitara definir nombres con más de una palabra, debe usarse guión bajo (" _ ") para separar las palabras.
- Los nombres de cualquier objeto de base de datos no debe empezar con (" _ ").
- Evitar nombrar campos que representan lo mismo de forma distinta. La forma en que se nombran iguales propiedades debe ser consistente en todo un esquema. Ejemplo: Nombrar al campo clave de la tabla Customer como Id, y después

referenciar en otras tablas como "customer_id", es incorrecto. El campo debe ser nombrado "id_customer" en todos los casos que se quiera almacenar una clave de Customer.

- Evitar tener columnas NULLABLES en una tabla. Esto es indicio de un esquema poco o nada normalizado. La falta de normalización puede conllevar problemas de consistencia en los datos en la medida que un mismo campo se puede terminar almacenando en varias tablas. Excesiva normalización puede tener asociada una pérdida de performance en ciertas operaciones sobre la base de datos. Es necesario encontrar el equilibrio correspondiente a los requerimientos de cada proyecto en este punto. Como regla general la tercera forma normal (3FN) es un buen punto intermedio.
- Evitar el uso de DISTINCT, ya que impacta en el rendimiento del motor de base datos, al realizar mucho I/O de Disco y Alto consumo de CPU
- Utilizar el TOP, cuando se desea traer un conjunto de registro.
- Evitar el uso de SELECT *, lo mejor es definir los campos que queremos devolver en nuestra consulta. También esto afecta en el rendimiento.
- Evitar el uso excesivo de la cláusula ORDER BY, se recomienda que la ordenación sea por el lado del cliente.
- Se debe utilizar la cláusula GROUP BY, con una función de agregación.
- Evitar en la cláusula WHERE, el uso de funciones numéricas, fechas y cadena, cuando compara una columna con una constante, ya que esto afecta el rendimiento en el motor de base de datos. Como por ejemplo:

```
select campo1, campo2 from tabla where WHERE SUBSTRING(campo1,1,5)='pollo'
```

```
select campo1, campo2 from tabla where WHERE YEAR(campo2)=2024'
```
- No utilizar el carácter comodín (%) al inicio del texto, con el operador LIKE en la cláusula WHERE para buscar un patrón específico en una columna. Esto genera un impacto negativo en el rendimiento del motor de base de datos.
- Cuando se inicia a programar en base de datos (procedimiento almacenados), las transacciones explícitas, deben iniciar con la instrucción BEGIN TRANSACTION y se termina explícitamente con una instrucción COMMIT o ROLLBACK. También se debe utilizar siempre el TRY.CATCH, para manejar errores, verificar @@TRANCOUNT antes de ROLLBACK. Estos procedimientos son poco comunes ya que normalmente el aplicativo maneja la transacción.
- Utilizar valores por defecto sobre las columnas en el cual el aplicativo o proceso de carga, no grabara un valor. Tener en cuenta que dicho valor por defecto debe estar alineado a las reglas del negocio.
- No usar IDENTITY, en su lugar utilizar Secuencia. Esta se puede utilizar a partir de la versión SQL SERVER 2012, hacia adelante.
- Es obligatorio que toda tabla debe tener Llave Primaria (PK).
- Se sugiere que los campos que conforman la llave primaria, deben ser aquellos que son utilizados por las consultas más frecuentes, ya que dichos campos están dentro de un índice tipo cluster. En SQL SERVER existen dos tipos de índices NO CLÚSTER y CLUSTER, este último solo se puede crear una por tabla, en cambio los índices NO CLÚSTER se pueden crear varias. Los índices CLUSTER son más rápidos que los índices NO CLÚSTER.
- Evitar tener tablas sin definición de Llave Foránea (FK), para controlar la integridad y consistencia de los datos.
- Evitar tener tablas sin índices CLÚSTER y NO CLÚSTER, más aún si estas tablas participan en un reporte y consultas de alto volumen de información.
- Evitar tener tablas u otros objetos innecesarios en la base de datos.
- Los tipos de datos usados en la creación de variables o parámetros durante el desarrollo de objetos de programación (procedimientos, funciones, paquetes, otros) deben coincidir con los tipos de datos de los campos de la tabla, el cual utilizaran.
- Las tablas que almacenan gran volúmenes de datos, deben estar particionadas. Una tabla particionada hace que las consultas sean más rápidas. La partición puede ser por año o por mes. También puede ser particionada por un conjunto de valores (lista).
- Deben de existir tablas históricas, esto con la finalidad de que en las tablas transaccionales o principales deben existir la información necesaria para las consultas del aplicativo. Esto ayudará mucho en los tiempos de respuestas de las consultas.
- El proceso de eliminación de una información a través de un sistema o aplicativo (api, servicio, microservicio, otros) no debe de ser una eliminación física del registro (soft delete), sino una actualización del registro para que no se visualice

en el sistema o aplicativo.

- Todo nuevo aplicativo o sistema debe contener un módulo de Mantenimiento, un módulo de Configuración y un módulo de Auditoría de DML (lenguaje de manipulación de datos - UPDATE, INSERT, DELETE), este con el objetivo de evitar que los cambios sobre las tablas, lo hagan a través de script con sentencia DML.
- No debe utilizarse Servidores Vinculados (Linked Server).

2.2 Nomenclatura de Objetos de base de datos

La longitud de los nombres de base de datos y esquemas debe tener como máximo 30 caracteres.

Los nombres de las bases de datos de los sistemas informáticos de Producción o de Desarrollo, deben hacer referencia al Proceso de Negocio que soporten. Para base de datos temporales, el nombre de la base restaurada debe incluir la fecha en que se tomó el backup. La fecha debe estar en el formato YYYYMMDD (Año, Mes y Día).

Una vez creada la base de datos para el sistema informático; y, en caso fuera necesario crear otra base de datos que está relacionado al mismo sistema informático, este creará un esquema debajo de la base de datos principal.

Por ejemplo:

La empresa Atlantic CITY, cuenta con la dirección de recursos humanos y este a su vez tiene tres (03) oficinas, Bienestar, Planilla y Marcación. Entonces la base de datos principal se llamará bd_recursoshumanos y sus esquemas sch_bienestar, sch_planilla y sch_marcación.

Base de Datos:

Se le debe anteceder el prefijo identificador de 2 caracteres “bd”, para indicar que es una base de datos, seguido del nombre del Proceso de Negocio. Para el ambiente de desarrollo, se le coloca al final el prefijo “dev”.

Tipo	Sintaxis	Ejemplos
Producción o QC	bd_<nombre proceso>	bd_promociones
Desarrollo	bd_<nombre proceso>_dev	bd_promociones_dev
Temporal	bd_<nombre proceso>_<yyyymmdd>	bd_promociones_20241106

Esquemas:

Se le debe anteceder el prefijo identificador de 3 caracteres “sch”, para indicar que es un esquema, seguido el nombre del Subproceso de negocio. No se debe utilizar el esquema por defecto DBO.

Tipo	Sintaxis	Ejemplos
Producción o QC	sch_<nombre sub_proceso>	sch_bienestar
Desarrollo	sch_<nombre sub_proceso>	sch_bienestar
Temporal	sch_<nombre sub_proceso>	sch_bienestar

2.3 Nomenclatura de Tablas.

El nombre de una tabla deberá ser lo más descriptivo posible y en singular, tratando de incluir las características más saltantes de la información que contendrá.

El nombre de la tabla debe tener un máximo de 30 caracteres, sin tilde, sin espacios en blanco y los nombres compuestos deben estar separados por guion bajo (_).

En el caso que sea un campo compuesto, sólo la última palabra significativa debe de ir en plural.

Se le debe anteceder el prefijo identificador de un carácter: "T", para indicar que es un Objeto tipo tabla.

En las tablas, a este prefijo les seguirá un identificador de tipo de tabla, de acuerdo a la siguiente nomenclatura:

Tipo de Tabla	Identificador
Maestro	M
Transacción	T
Histórico	H
Auditoría	A
Temporal	X

Solo para la tabla de auditoría, después de colocar el identificador de tipo de tabla, se colocará el nombre de la instancia a la cual pertenece la data auditada.

También, una vez ingresada el prefijo y el identificador, se debe agregar la abreviatura del propósito de la tabla (Ejemplo Invitados = invt, juegos = jueg) y finalmente el nombre de la tabla.

Por ejemplo:

tm_invt_invitado = Tabla maestra de invitados

tt_opro_operacion = Tabla transaccional de las operaciones de los invitados

tt_movt_movimiento = Tabla transaccional de los movimientos de los invitados.

th_opro_operacion = Tabla histórica de las operaciones de los invitados.

ta_proddb07_bd_promocion = Tabla de auditoría DML sobre la base de datos promoción, la cual pertenece a la instancia PRODDB07. Por seguridad, dicha tabla se aloja en otra instancia.

tx_grp_grupo_jugadores = Tabla temporal de los grupos de invitados.

Para una base de datos tipo datamart o datawarehouse, el nombre de las tablas con el rol de dimensión tendrá el prefijo dim, seguido del nombre, y aquellos que tienen el rol de hechos tendrá el prefijo fact, seguido del nombre.

Por ejemplo:

dim_articulo

dim_cartera

fact_juegos

fact_pedido

2.4 Nomenclatura de Campos.

Los nombres de los campos de una tabla deben ser en singular, simples, representativos e intuitivos.

Los campos que forman parte de una llave primaria deben ser los primeros campos en la tabla.

Los nombres con más de una palabra deben separar las palabras por guión bajo (_).

En el caso que sea un campo compuesto, sólo la última palabra significativa debe de ir en plural.

La longitud de los nombres de campos debe tener como máximo 30 caracteres y en singular.

Cuando se utilicen varias tablas en una consulta y se presenten campos ambiguos, se tendrá que utilizar Alias sobre las tablas.

En caso de que el campo o columna sea parte de un “foreign key”, entonces el nombre del campo será el mismo que el de la tabla de origen con la que se relaciona el campo.

La sintaxis del nombre del campo tendrá el siguiente formato

```
1 <AbrevTabla><Abrevtype><Nombrecampo>
2
3 <AbrevTabla> = Abreviatura de la tabla, la cual pertenece al campo.
4
5 <Abrevtype> = Abreviatura que indica el Tipo de datos.
6
7 char = c
8
9 varchar = v
10
11 nvarchar = nv
12
13 datetime = dt
14
15 <Nombrecampo> = Descripción clara que identifica el campo.
```

Ejemplo:

invt_c_nombre

invt_dt_fechanacimiento

Así mismo, cada tabla tendrá que presentar un campo que indica el estado de registro; es decir, si es persistente o no persistente. Esto se utilizará, para que no se realice una eliminación física del registro (soft delete). El nombre de ese campo se denominará “flg_persistente” de tipo de dato “bit” (1 = verdadero, 0 = falso), que significa estado del registro.

Todas las tablas deben tener sus siete (07) campos de auditoría:

Campos	Descripción	Tipo de dato
usuario_registro	Usuario que creó el registro.	Varchar(30)
fecha_registro	Fecha en que se creó el registro.	datetime
usuario_modificacion	Usuario que modificó el registro.	Varchar(30)
fecha_modificacion	Fecha que se modifica el registro.	datetime
ip	Registra la IP del usuario que hizo el cambio	varchar(30)
hostname	Registra el nombre del host del usuario que hizo el cambio.	varchar(50)
flg_persistente	Flag de eliminación lógica del registro.	bit

2.5 Nomenclatura de Vista.

El nombre de una vista deberá ser lo más descriptivo posible, tratando de incluir las características más saltantes de la información que contendrá.

El nombre de una vista no debe tener espacio en blanco, en caso sea un nombre compuesto debe utilizar el guion bajo “_”.

La longitud de los nombres debe tener como máximo 30 caracteres.

Se le debe anteceder el prefijo identificador de 3 caracteres: uvw.

Sintaxis	Ejemplo
uvw_<nombre_vista>	uvw_jugadores_fisicos

2.6 Nomenclatura de Funciones.

El nombre de una función deberá ser lo más descriptivo posible, tratando de incluir las características más saltantes de la información que contendrá.

El nombre de una función no debe de tener espacio en blanco, en caso sea un nombre compuesto debe utilizar el guion bajo “_”.

La longitud de los nombres debe tener como máximo 30 caracteres.

Se le debe anteceder el prefijo identificador de 3 caracteres ufn, para indicar que es una función valor escalar, y tfn para indicar que es una función valor tabla.

Tipo	Sintaxis	Ejemplo
Valor Escalar	ufn_<nombre_función>	uvw_fecha <i>diacasino</i>
Valor Tabla	tfn_<nombre_función>	tfn_autoexclusion

2.7 Nomenclatura de Procedimiento Almacenados

El nombre de un Procedimiento Almacenado deberá ser lo más descriptivo posible, tratando de incluir las características más saltantes de la información que contendrá.

El nombre de una función no debe de tener espacio en blanco, en caso sea un nombre compuesto debe utilizar el guion bajo “_”.

La longitud de los nombres debe tener como máximo 30 caracteres.

Se le debe anteceder el prefijo identificador de 3 caracteres usp, para indicar que es un procedimiento de usuario, ya que si colocamos sp, el motor de base de datos buscará primero en el catálogo del sistema (master).

Sintaxis	Ejemplo
usp_<nombre_procedimiento>	usp_obtener_invitado_ganadores_torneo_calí maco
	usp_obtener_jugador_lobbie

2.8 Nomenclatura de Secuencias

El nombre de una secuencia debe estar relacionado con el nombre de la tabla, a la cual se le adjuntará. Por ejemplo, si la tabla se llama tt_opro_operaciones, la secuencia se llamaría useq_operaciones.

El nombre de una secuencia no debe de tener espacio en blanco, en caso sea un nombre compuesto debe utilizar el guion bajo “_”.

La longitud de los nombres debe tener como máximo 30 caracteres.

Se le debe anteceder el prefijo identificador de 4 caracteres useq, para indicar que es una secuencia de usuario.

Sintaxis	Ejemplo
useq_<nombre_secuencia>	useq_operaciones
	useq_movimientos

2.9 Nomenclatura de Tipo de Dato y Tipo de Tabla definido por el usuario.

El nombre de un tipo de datos deberá ser lo más descriptivo posible, tratando de incluir las características más saltantes de la información que contendrá.

El nombre de un tipo de datos no debe de tener espacio en blanco, en caso sea un nombre compuesto debe utilizar el guion bajo “_”.

La longitud de los nombres debe tener como máximo 30 caracteres.

Se le debe anteceder el prefijo identificador de 4 caracteres uddt, para indicar que es un tipo de datos definido por usuario, y udt para indicar que es un tipo de tabla definido por un usuario.

Tipo	Sintaxis	Ejemplo
Valor Dato	<uddt>_<nombre_tipodatos> >	uddt_nuevoalias
Valor Tabla	<udtt>_<nombre_tipotabla>	udtt_grillaparametro

2.10 Nomenclatura de Triggers.

El nombre del trigger deberá ser lo más descriptivo posible, tratando de incluir las características más saltantes de la información que contendrá.

El nombre del trigger no debe de tener espacio en blanco, en caso sea un nombre compuesto debe utilizar el guion bajo “_”.

La longitud de los nombres debe tener como máximo 30 caracteres.

Se le debe anteceder el prefijo de 3 caracteres, TRG, para indicar que es un trigger.

Así mismo, los triggers se clasifican en tres (03) tipos: DML, DDL, LOGON.

Para los trigger tipo DML, en la parte final de su nombre, se le indicará un identificador:

Identificador	Descripción
UPD	Acción de Actualización
INS	Acción de Inserción
DEL	Acción de Eliminación
ALL	Realiza UPD y INS

Nota Importante:

El trigger que valida, el evento de eliminación, sólo se aplicará cuando el usuario realice la eliminación del registro, de manera directa a la base de datos; es decir, no se deberá realizar a través del aplicativo o sistema.

Por ejemplo:

Tipo	Sintaxis	Ejemplo
DML	<trg>_<dml>_<tabla>_<identificador>	trg_dml_tt_eventoregistro_upd
		trg_dml_tt_eventoregistro_ins
DDL	<trg>_<ddl>_<nombreevento>	trg_ddl_capturacambios
LOGON	<trg>_<log>_<nombreevento>	trg_log_listablanca

2.11 Nomenclatura de Usuarios de base de datos

El nombre del usuario de base de datos deberá ser lo más descriptivo posible, tratando de incluir las características relacionadas al proceso de negocio.

La longitud de los nombres debe tener como máximo 45 caracteres.

Se le debe anteceder el prefijo de 3 caracteres, APP, para indicar que es un usuario de base de datos para una aplicación o carga de trabajo.

Luego de agregar el prefijo APP, se debe colocar el identificador, para indicar que tipo de tecnología o herramienta es.

Identificador	Descripción
ETL	Usuario que será utilizado para un proyecto Extracción, Transformación y Carga de Datos.
WEB	Usuario que será utilizado para una aplicación WEB
API	Usuario que será utilizado para una aplicación tipo api.
WS	Usuario que será utilizado por un servicio web.
DSKT	Usuario que será utilizado para una aplicación tipo escritorio.
MS	Usuario que será utilizado por un microservicio.
DATAPROC	Usuario que será utilizado para proyectos de Ingesta de datos en Nube GCP
LS	Usuario que será utilizado por un Link Server, para aplicaciones legacy.
PB	Usuario que será utilizado por Power BI
OTROS	Para otros propósitos.

Ejemplos:

Tipo	Sintaxis	Ejemplo
------	----------	---------

ETL	app_etl_hostOrigen_hostDest_r_proceso	app_etl_nodo1_p01_r_SorteoCalimaco
	app_etl_hostOrigen_hostDest_r_proceso r=lectura	app_etl_p07_p01_r_SorteoCalimaco
	app_etl_hostOrigen_hostDest_rw_proceso w=escritura	app_etl_p07_p01_rw_SorteoCalimaco
API	app_api_hostOrigen_r_proceso	app_api_p07_r_sorteoestelario
	app_api_hostOrigen_rw_proceso	app_api_p07_rw_sorteoestelario
WEB	app_web_hostOrigen_proceso	app_web_p03_crmcloud
WS	app_ws_hostOrigen_proceso	app_ws_p02_calculopuntos
DSKT	app_dskt_hostOrigen_proceso	app_dskt_2k8_notarios
MS	app_ms_hostOrigen_proceso	app_ms_p07_generaopciones
DATAPROC	app_dataproc_hostOrigen_hostDest_r_proceso r=lectura	app_dataproc_stag_gcp_r_gastosprom
LS	app_ls_hostOrigen_hostDest_proceso	app_ls_p01_p07_calculocuentas

2.12 Nomenclatura de Tipo de Constraints.

El nombre de los constraints debe tener un máximo de 30 caracteres. Los Constraints permiten las restricciones, aplicando reglas a nivel de tablas. También las restricciones evitan las eliminaciones de registros en tablas hijos o tabla Maestra, con la finalidad de conservar la Integridad de los datos.

Asimismo, los tipos de Constraints son los siguientes:

Tipo de Constraint	Sintaxis	Ejemplo
PRIMARY KEY	PK_NOMTABLA	pk_tt_invexcl_invitadosexcluidos
FOREING KEY	FK_TABLAHIJO_TABLAPADRE	fk_tm_ditto_distrito_tt_invexcl_invitadosexcluidos
NOT NULL	NN_NOMTABLA_CAMPO	nn_tm_ditto_distrito_codigo
CHECK	CK_NOMTABLA_CAMPO	ck_tt_inv_invitados_sexo
UNIQUE	UQ_NOMTABLA_CAMPO	uq_tm_jue_juegos_codigo

2.13 Nomenclatura de Jobs

El nombre del job deberá ser lo más descriptivo posible, tratando de incluir las características relacionadas al proceso de negocio.

La longitud de los nombres debe tener como máximo 45 caracteres.

Se le debe anteceder el prefijo de 3 caracteres, JOB, para indicar que es job.

Luego de agregar el prefijo JOB, se debe colocar el identificador, para indicar que tipo de trabajo que va a realizar.

Identificador	Descripción
T-SQL	El job ejecuta un script t-sql.
CMD_EXEC	El job ejecuta comandos del sistema operativo o programa ejecutable.
POWERSHELL	El job ejecuta un script power shell.
SSIS	El job ejecuta paquetes de integration services.
ANALYSISCOMMAND	El job ejecuta el comando de Analysis Services (XMLA).
ANALYSISQUERY	El Job ejecuta consulta de Analysis Service (MDX, DMX)
OTROS	El job ejecuta otros comandos.

Ejemplos:

Tipo	Sintaxis	Ejemplo
T-SQL	job_tsq1_procesonegocio	job_tsq1_puntos_ad
SSIS	job_ssis_procesonegocio	job_ssis_navidad_grupousuarios

Nota Importante:

Para nombrar los jobs que ejecutan un ETL, por parte del equipo de DM (data management) y los ETL que se despliega en un pase a producción, en ambos casos la nomenclatura del nombre del job, será bajo los lineamientos que DM está ejecutando.

2.14 Nomenclatura de Índices

Los índices son objetos de la base de datos, cuya función es optimizar el acceso a datos facilitando la recuperación de datos, permitiendo el acceso directo y acelerando las búsquedas, consultas y otras operaciones que mejoran el rendimiento general. A medida que las tablas se van haciendo más grandes y se desea hacer consultas sobre estas tablas, los índices son indispensables.

La longitud de los nombres debe tener como máximo 30 caracteres.

Se le debe anteceder el prefijo de tres caracteres, IDX, para indicar que es índice.

Luego de agregar el prefijo IDX, se debe colocar el identificador, para indicar que tipo de índice.

Identificador de Índice	Descripción
-------------------------	-------------

C	Índice Clúster
CUV	Índice Clúster para vista
D	Índice No Clúster Duplicado
U	Índice No Clúster Único.
F	Índice No Clúster Filtrado.
C	Índice No Clúster Comprimido.
I	Índice No Clúster Duplicado con Include.

Ejemplos:

Tipo	Sintaxis	Ejemplo
C	idx_c_nombretabla	idx_c_tt_operacion
D	idx_d_nombretabla_nro.	idx_d_tt_movimiento_01
U	idx_u_nombretabla_nro.	idx_u_tm_juegos_01
F	idx_f_nombretabla_nro.	idx_f_tt_grupojugadores_03
C	idx_c_nombretabla_nro.	idx_c_tm_distrito_02
I	idx_i_nombretabla_nro.	idx_i_tt_premios_01

3. DOCUMENTACIÓN

La documentación que debe de presentar la programación de objetos como Stored Procedures, View, Functions, Triggers, y Otros, en las bases de datos es muy importante para la gestión de cambios, es por ello que debemos tener en consideración lo siguiente:

3.1 Documentación de Bloque de Código

Considerar el siguiente formato para la cabecera en la creación de los objetos:

```
/*-----
```

Nombre: Nombre del procedimiento, función, vista, trigger, entre otros.

Descripción: Breve descripción de lo que hace.

Autor: Nombres y Apellidos del programador

Fecha de creación: Fecha de la creación del objeto.

Proyecto: Nombre del proyecto.

Versión: Versión del proyecto.

Parámetros: Indica los parámetros que utilizarán.

Servidor y BD: Nombre de Instancia y Base de Datos.

```
-----*/
```

Ejemplo:

```

1  /*-----
2
3  Nombre: [sch_survey].[usp_get_surveyviews_by_id_jugador_online]
4
5  Descripción: Obtener la fecha de visualización y el interceptor de la encuesta por jugador y en un rango de fecha
6
7  Autor: Carlos Castro de la Cruz
8
9  Fecha de creación: 04/12/2024
10
11 Proyecto: Encuesta QUALTRICS.
12
13 Versión: 1.0
14
15 Parámetros: [sch_survey].[usp_get_surveyviews_by_id_jugador_online] '20345'
16
17 Instancia - BD - Esquema: PRODDB01 - [db_api_invitados_casino_online] - [sch_survey]
18
19 -----*/
20
21 A continuación de la cabecera general, se deberá incluir la cabecera del control de cambios:
22
23 {
24
25 @Nro. Fecha Autor Descripción
26
27 }

```

Ejemplo:

```

1  @1 06/12/24      ccastro Habilitar el estado del jugador para sorteo.
2
3  WHERE A.estado = 1 - @1
4
5  AND B.estado = 1 - @1
6
7  AND C.estado = 1
8
9  ORDER BY B.id_etapa asc

```

3.2 Documentación para diccionario de datos

Los comentarios deben ser incluidos en la creación física de las base de datos, sobre todo para cambios de estructura realizadas directamente en la base de datos física.

Luego se podrá actualizar el modelo de datos y diccionario de datos haciendo Ingeniería Inversa usando el CASE de modelamiento.

Ejemplo para colocar una descripción a la tabla:

```

1  EXEC sp_addextendedproperty
2
3      @name = N'MS_Description',
4
5      @value = N'Tabla mae aplicacion',
6
7      @level0type = N'SHEMA',
8
9      @level0name = N'sch_gespago',

```

```

10
11     @level1type = N'TABLE',
12
13     @level1name = N'mae_aplicacion',

```

Ejemplo para colocar una descripción a una columna:

```

1 EXEC sp_addextendedproperty @name = N'MS_Description',
2
3     @value = N'Identificador único de la aplicación.',
4
5     @level0type = N'SHEMA',
6
7     @level0name = N'sch_gespago',
8
9     @level1type = N'TABLE',
10
11     @level1name = N'mae_aplicacion',
12
13     @level2type = N'COLUMN',
14
15     @level2name = N'id_aplicacion';

```

3.3 Documentación para parámetros, variables y constante

Los nombres de variables, parámetros y constantes deben ser en singular, simples, representativos e intuitivos.

Los nombres con más de una palabra deben separar las palabras por guion bajo (_).

La longitud de los nombres de variables, parámetros y constantes debe tener como máximo 30 caracteres.

La nomenclatura de un parámetro o una variable debe tener la siguiente sintaxis:

```

1 <in o out>_ <parámetro o variable> _<tipo_de_dato>_<nombre>
2
3 Donde:
4
5 <in o out>    in = entrada
6
7                out = salida
8
9 <parámetro o variable>  p=parámetro
10
11                        v=variable
12
13 <tipo_de_dato>
14
15                        v=varchar
16
17                        n = number
18
19                        d = date
20
21                        dt = datetime
22
23                        c= char
24
25                        d= decimal
26
27 <nombre>    Debe estar en singular, simple, representativo e intuitivo.

```

Ejemplo:

```
1 in_p_c_codinvitado = Parámetro de entrada con tipo de datos char, y almacena código invitado.  
2  
3 out_p_n_puntos = Parámetro de salida con tipo de dato numérico, y almacena los puntos.  
4  
5 in_v_n_codgrpjuego = Variable de entrada con tipo de dato numérico y almacena código grupo de juego.  
6  
7 out_v_d_cumpleaños = Variable de salida con tipo de dato fecha y almacena la fecha de nacimiento.  
8  
9 out_v_n_nrodias = Variable de salida con tipo de dato numérico y almacena numero de días.
```

La nomenclatura de una constante (variable que no tendrá cambio durante la ejecución del programa, ya sea un procedimiento almacenado o función) es como sigue:

<@>_<NombreMayuscula>

Ejemplo:

@CODJUEGO=101